Advanced C Programming Course - Jason Fedin

David Corzo

21 febrero 2023 02:20

Contents

Chapter 1

Introduction

1.1 Welcome to class

- This is the second part of the C programming course for begginers.
- After taking this class you should be an expert in C.
- The last class is a pre-requisite to this one. If you are a begginer this course is not for you.
- Topics:
  - Storage classes: auto, register, static, and extern.
  - Advanced data types: typedef, variable length arrays, flexible array numbers, complex number types.
  - Type qualifiers: const, volatile, and restrict.
  - Bit manipulation:
    * Binary numbers and bits
    * Bitwise operators (logical and shifting).
    * Bitmasks and bitfields.
  - Advanced control flow: goto, null, comma operator, setjmp and longjmp.
  - More on input and output: getchar, putchar, fgets, puts, sprint, fprintf, fflush, etc.
  - Advanced function concepts: veriadic functions (variable number of arguments to a function), recursive functions, inline functions.
  - unions: overview, defining and accedding union members.
  - Advanced preprocessor concepts: #define, #pragma, #error, #, ##. Conditional compilation (#ifdef, #endif, #else, #elif, #undef, etc). Include guards.
  - Macros: overview (vs. functions when to use). Predefined macros, creating your own macros.
  - Advanced debugging and compiler flags:
    * Debugging with the pre-processor, more on gdb.
    * Core files, getting the stack trace.
    * Static analysis and profiling.
  - Static libraries and shared objects: overview, creation, dynamic loading.
  - Working with larger programs: dividing your program into multiple files and compiling multiple files.
  - Advanced pointers: double pointers (pointers to pointers), function pointers, more on void parameters.

  - Useful C libraries: the assert library, general utilities library (stdlib.h), (exit, atexit, qsort, memcpy, abort), date and time functions.
  - Data structures: linked lists, stacks, queues, and trees.
  - Inter-process communication (IPC) (unix based using cygwin): overview (message queues, shared memory, piping), forking and signals.
  - Threads (pthread (posix), not `<threads.h>` (mainly because it's not portable) from C11): overview, creating a thread, mutexes, and semaphores, thread management (multi-threading, join, detach).
  - Networking (unix based using cygwin): overview (client/server model), creating server and client sockets.

- Expectations:
  - Many challenges, solutions, and examples.
  - Practicing theory in real time, lots of coding.
  - Ability to write advanced C programs.
  - You will be able to write efficient, high quality code that is modulat, lowly coupled (low dependencies).
  - Master the art of problem solving in programming using efficient, proven methods.
  - You will understand advanced concepts of C.
  - And have fun.

1.2 Class organization

- This class is not a tutorial, or a how to, its focused on the why:
  - This with the purpose of having a genuine learning experience.
  - Understand the programming language as a whole.
  - Most udemy courses are not like this.
- Powerpoint slides:
  - This is the way the instructor found best to demonstrate code and abstract concepts.
  - The instructor will not just be reading off of slides.
    * He will be proving insight through experience.
    * Providing through explanations.
- Demonstrations:
  - Code examples in an IDE.
  - Applying the theory.
- Challenges (coding challenges):
  - All challenges will have the solutions, but try first without looking at the solution.

1.3 The C99 Standard

+

- Overview:
  - C has several standards (C89, C99, C11).

  - We already covered concepts of the C99 standard, but in the begginer course we maily focused on C89, the only parts of C99 we touched on were:
    * bool data type in stdbool.h.
    * Variable length arrays.
    * Single line comments.

- This class will include more C99 functionality.

1.3.1 History

- C99 is a revised standard for the C programming language. the "99 in the name stads for the year it came out.
- C99 has not been widely adopted:
  - Not all popular C compilers support it.
  - Of all the compilers that offer C99 support, most support only a subset of the language. Partially supported.
- This is why the begginer course focused on C89. C89 is almost universally supported where as C99 is not.

1.3.2 C99 features

- The features that we will be looking at will be:
  - macros with a variable number of arguments.
  - The added data types: _Complex (floating points for complex number), _Imaginary (imaginary floating numbers.)
  - Designated initializers: initialize elements of an array, union, or struct explicitly by subscript or name.
  - Restricted pointers: type qualifier only used on pointers.
  - New comment sintax: `//` the double slash.
  - Inline functions: supplies a hint for the compiler to perform optimizations.
  - Variable length arrays: in C89 the array size has to be declared, C99 permits declaration of array sizes using an integer variable or any valid integer expression.
  - Flexible array members: declare an array of unspecified length as the last member of a struct.
  - Declaration of members: now its legal to declare variables at any point in the program, so instead of `int i; for (i = 0; i < 10; i++) {}` you can declare the variable in the loop itself `for (int i = 0; i < 10; i++) {}` this is an additional feature included in C99. Better because they are local and cannot be accidentaly misused after the loop ends if they were going to be used only in the loop.

1.4 The C11 Standard

- We have not seen C11 concepts.
- Features added to C11:
  - Multithreading support, safer standard libraries, better compliance with other industry standards.
- Fixing issues of C99:

  - Some mandatory features are optional (variable length arrays and complex types).
  - Basically if you want to use variable length arrays, the compiler will optionally include this feature, so if you use C11 compilers be ware that some C99 features are not always included.
- C11 focuses on better compatibility with C++ as much as possible.
- We may touch on some concepts of C11 but it will be minimal.
- C11 standarizes many features that have already been available in common compiler implementations.
- Defines a memory model that better suits multithreading.
- Fixes problems with the C99 standard:
  - Some of its mandatory features proved difficult to implement in some platforms.
  - Politics: cooperation with C and C++ standards commitees in the 90s was lacking, C11 is more compatible.
  - Design mistakes of C99 are fixed in C11.
- C11 focuses on security (string manipulation functions, input/output):
  - A new set of safer standard functions that aim to replace the traditional unsafe funtions:
    * bounds checking functions: begin and end in _s (strcat_s, strcpy_s, etc).
    * Removal of the "gets functions. This was an unsafe function and got people hacked.
    * C11 has optional to the standard features (will not focus on due to most compilers supporting it).
- Topics pending:
  - Support for anonymous structs and unions:
    * Has neither a tag name nor a typedef name.
    * Useful when unions and structures are nested.
  - Static assertions:
    * Evaluated during translation at a later phase than #if and #error.
    * Let you catch errors that are impossible to detect during the preprocessing phase.
  - No-return functions:
    * Declares a function that does not return.
    * Suppresses compiler warnings on a function that doesn't return.
    * Enables certain optimizations that are allowed only on functions that don't return.
  - Multithreading:
    * The biggest change in C11 is its standardized multithreading support.
    * C has supported multithreading for decades:
      · However, all of the popular C threading libraries have thus far been non-standard extensions and hence not-portable.
    * The new C11 header file `<threads.h>` declares functions for creating and managing threads, mutexes, condition variables.
    * However it is not supported on windows, and thus we will learn `<pthread.h>` instead (which is posix compliant).